

homework2

2026-09-06

Homework 2

Question 1

a) The 200-run results are shown below. Training MSE was nonincreasing in every run.

m	Average train	Expected train	Average test	Expected test
0	1.3147	1.2974	1.3367	1.3174
1	1.1881	1.1763	1.2279	1.2163
2	1.0986	1.0872	1.1546	1.1472
3	1.0415	1.0319	1.1144	1.1119
4	1.0105	1.0022	1.1046	1.1022
5	0.9806	0.9750	1.0979	1.0950
6	0.9623	0.9547	1.0940	1.0947
7	0.9481	0.9395	1.0995	1.0995
8	0.9327	0.9243	1.1034	1.1043
9	0.9173	0.9105	1.1096	1.1105
10	0.9061	0.8992	1.1186	1.1192
11	0.8948	0.8878	1.1251	1.1278
12	0.8846	0.8764	1.1325	1.1364
13	0.8744	0.8640	1.1390	1.1440
14	0.8625	0.8524	1.1489	1.1524
15	0.8524	0.8419	1.1594	1.1619
16	0.8409	0.8315	1.1675	1.1715
17	0.8279	0.8209	1.1786	1.1809
18	0.8166	0.8106	1.1910	1.1906
19	0.8052	0.8002	1.2052	1.2002
20	0.7948	0.7900	1.2170	1.2100

b) The theoretical values in the table use

$$b_m^2 = \frac{1}{n} \|(I - H_m)\mu\|^2,$$
$$E(\text{MSE}_{\text{train},m} \mid X_{\text{all}}) = b_m^2 + 1 - \frac{m+1}{100},$$
$$E(\text{MSE}_{\text{test},m} \mid X_{\text{all}}) = b_m^2 + 1 + \frac{m+1}{100}.$$

The simulation averages are close to the expectations. Both test-error curves have their minimum at $m = 6$.

c) A single test curve contains more random variation, while averaging 200 runs reduces that variation. Adding predictors lowers approximation bias but increases estimation variance, so test error can rise even though training error cannot.

```
set.seed(43202)
n <- 100
p <- 20
R <- 200
X_all <- matrix(rnorm(n * p), nrow = n)
beta <- 0.4^sqrt(1:p)
mu <- drop(X_all %*% beta)

# QR bases are fixed across all repetitions.
```

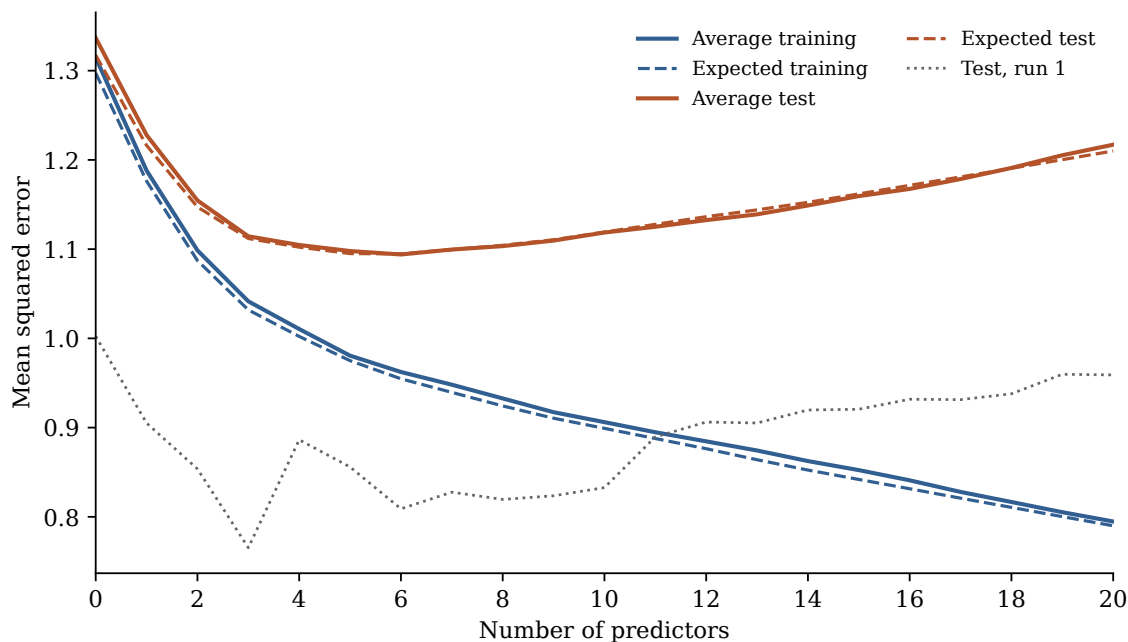


Figure 1: Training and test MSE for the fixed design.

```
Q <- lapply(0:p, function(m) {
  X <- cbind(1, X_all[, seq_len(m), drop = FALSE])
  qr.Q(qr(X))
})
project <- function(q, y) drop(q %*% crossprod(q, y))
train <- test <- matrix(NA_real_, R, p + 1)
for (k in 1:R) {
  y <- mu + rnorm(n)
  y_star <- mu + rnorm(n)
  for (m in 0:p) {
    y_hat <- project(Q[[m + 1]], y)
    train[k, m + 1] <- mean((y - y_hat)^2)
    test[k, m + 1] <- mean((y_star - y_hat)^2)
  }
}

bias_mse <- sapply(Q, function(q) {
  mean((mu - project(q, mu))^2)
})
d <- (1:(p + 1)) / n
results <- data.frame(
  m = 0:p,
  train = colMeans(train),
  E_train = bias_mse + 1 - d,
  test = colMeans(test),
  E_test = bias_mse + 1 + d
)
monotone <- all(apply(train, 1, function(z) {
  all(diff(z) <= 1e-10)
}))
print(round(results, 4))
print(monotone)
print(c(average = which.min(results$test) - 1,
  expected = which.min(results$E_test) - 1))

matplot(0:p, cbind(results[, 2:5], test[1, ]),
  type = "l", lty = c(1, 2, 1, 2, 3), lwd = 2,
  col = c("#315F91", "#315F91", "#B4532A", "#B4532A", "gray45"),
```

```

xlab = "Number of predictors", ylab = "Mean squared error")
legend("topright", bty = "n", cex = 0.8,
  legend = c("Average training", "Expected training",
    "Average test", "Expected test", "Test, run 1"),
  col = c("#315F91", "#315F91", "#B4532A", "#B4532A", "gray45"),
  lty = c(1, 2, 1, 2, 3), lwd = 2)

```

Question 2

a) The target mean is

$$\mu_0 = 0.5 + 0.5 = 1.$$

m	Squared bias	Variance	Expected error	Simulation average
0	1.0000	0.0100	1.0100	1.0189
1	1.0000	0.0100	1.0100	1.0189
2	0.2500	0.0200	0.2700	0.2671
3	0.2500	0.0200	0.2700	0.2671
4	0.2500	0.0200	0.2700	0.2671
5	0.0000	0.0300	0.0300	0.0338
6	0.0000	0.0300	0.0300	0.0338

Omitted predictors create squared bias, while estimating the intercept and included slopes adds variance. The error changes at $m = 2$ and $m = 5$ and is unchanged at the other additions.

b) The simulation averages are included in the table and plotted against the theoretical values. The smallest minimizer is $m = 5$, with expected squared error 0.03; $m = 6$ gives the same error.

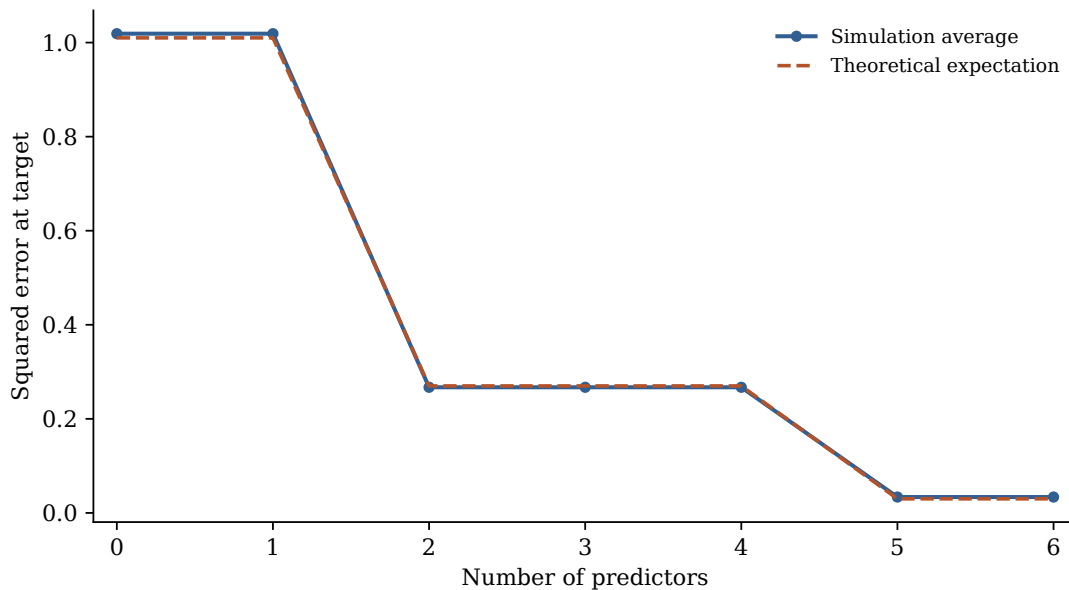


Figure 2: Simulation and theoretical squared error at the target.

```

set.seed(43203)
n <- 100
p <- 6
R <- 200
X_all <- outer(1:n, 1:p, function(i, j) {
  sqrt(2) * cos(pi * j * (i - 0.5) / n)
})

```

```

})
beta <- rep(0.5, p)
x0 <- c(0, 1, 0, 0, 1, 0)
mu <- drop(X_all %*% beta)
mu0 <- sum(x0 * beta)

errors <- matrix(NA_real_, R, p + 1)
for (k in 1:R) {
  y <- mu + rnorm(n)
  for (m in 0:p) {
    X <- cbind(1, X_all[, seq_len(m), drop = FALSE])
    b <- lm.fit(X, y)$coefficients
    prediction <- sum(c(1, x0[seq_len(m)]) * b)
    errors[k, m + 1] <- (prediction - mu0)^2
  }
}

squared_bias <- variance <- numeric(p + 1)
for (m in 0:p) {
  omitted <- if (m < p) (m + 1):p else integer(0)
  squared_bias[m + 1] <- sum(x0[omitted] * beta[omitted])^2
  variance[m + 1] <- (1 + sum(x0[seq_len(m)]^2)) / n
}
results <- data.frame(m = 0:p, bias = squared_bias,
  variance = variance, expected = squared_bias + variance,
  simulated = colMeans(errors))
print(round(results, 4))
print(c(first_minimum = which.min(results$expected) - 1))

plot(0:p, results$simulated, type = "o", pch = 16,
  col = "#315F91", xlab = "Number of predictors",
  ylab = "Squared error at target", lwd = 2)
lines(0:p, results$expected, lty = 2, lwd = 2, col = "#B4532A")
legend("topright", bty = "n", cex = 0.85,
  legend = c("Simulation average", "Theoretical expectation"),
  col = c("#315F91", "#B4532A"), lty = c(1, 2), lwd = 2)

```

- c) Only predictors 2 and 5 contribute because the other coordinates of x_0 are zero. Average test MSE considers the entire design, so a predictor can matter across the observations even when it has no effect at this particular target.

Question 3

- a) Let $P = I - H$ and $d = p + 1$. Then

$$\begin{aligned}
 y - Hy &= P\mu + P\epsilon, \\
 E(\|y - Hy\|^2 \mid X) &= \|P\mu\|^2 + \sigma^2 \text{tr}(P) \\
 &= B^2 + \sigma^2(n - d), \\
 E(\text{MSE}_{\text{train}} \mid X) &= \frac{B^2}{n} + \sigma^2 \left(1 - \frac{d}{n}\right).
 \end{aligned}$$

For the independent test response,

$$\begin{aligned}
 y^* - Hy &= P\mu + \epsilon^* - H\epsilon, \\
 E(\|y^* - Hy\|^2 \mid X) &= B^2 + n\sigma^2 + \sigma^2 \text{tr}(H) \\
 &= B^2 + \sigma^2(n + d), \\
 E(\text{MSE}_{\text{test}} \mid X) &= \frac{B^2}{n} + \sigma^2 \left(1 + \frac{d}{n}\right).
 \end{aligned}$$

All cross terms have expectation zero by the mean-zero and independence assumptions.

b) The expected optimism is

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid X) = \frac{2(p+1)\sigma^2}{n}.$$

For $n = 120$, $p = 4$, $B^2/n = 0.04$, and $\sigma^2 = 1$:

$$E(\text{MSE}_{\text{train}} \mid X) = 0.04 + 1 - \frac{5}{120} = 0.998333,$$

$$E(\text{MSE}_{\text{test}} \mid X) = 0.04 + 1 + \frac{5}{120} = 1.081667,$$

$$\text{Expected optimism} = \frac{10}{120} = 0.083333.$$

These are averages, so an individual test MSE can still be lower than its corresponding training MSE.

c) The corrected estimate and Mallows' C_p are

$$\widehat{\text{MSE}}_{\text{test}} = \frac{116.4 + 2(5)(0.96)}{120} = 1.05,$$

$$C_p = \frac{116.4}{0.96} - 120 + 10 = 11.25.$$

Their relationship is

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\hat{\sigma}^2}{n}(C_p + n) = \frac{0.96}{120}(11.25 + 120) = 1.05.$$

With common n and $\hat{\sigma}^2$, this is an increasing transformation, so both criteria select the same model.

Question 4

a) The model fits and common variance estimate are

Model	p	RSS	Residual df
A	7	1,098,908.566	362
B	8	1,089,717.057	361
Full	10	1,089,451.721	359

$$\hat{\sigma}^2 = \frac{1,089,451.721}{370 - 11} = 3,034.684460.$$

b) The criterion values are

Model	C_p	Reduced AIC	Reduced BIC
A	8.116	2974.640	3005.948
B	7.087	2973.532	3008.754

C_p and AIC select Model B; BIC selects Model A.

c) Adding **s6** reduces RSS by 9,191.509. The fit improvements and additional penalties are

$$\text{AIC/BIC fit improvement} = 370 \log \left(\frac{\text{RSS}_A}{\text{RSS}_B} \right) = 3.108,$$

$$\text{AIC penalty} = 2,$$

$$\text{BIC penalty} = \log(370) = 5.914,$$

$$C_p \text{ fit improvement} = \frac{\text{RSS}_A - \text{RSS}_B}{\hat{\sigma}^2} = 3.029 > 2.$$

The improvement exceeds the AIC and C_p penalties but not the BIC penalty, explaining why BIC prefers the smaller model. Selection only identifies the preferred candidate under a criterion; it does not prove that the model generated the data.

```
data <- read.csv("data/diabetes.csv")[1:370, ]
n <- nrow(data)
A <- c("bmi", "bp", "s5", "sex", "s1", "s2", "s4")
B <- c(A, "s6")
Full <- c("age", "sex", "bmi", "bp", "s1", "s2",
         "s3", "s4", "s5", "s6")
models <- list(A = A, B = B, Full = Full)

rss <- sapply(models, function(vars) {
  X <- cbind(1, as.matrix(data[vars]))
  sum(lm.fit(X, data$y)$residuals^2)
})
p <- sapply(models, length)
df_full <- n - p["Full"] - 1
sigma2 <- rss["Full"] / df_full

results <- data.frame(model = names(models), p = p,
                      RSS = as.numeric(rss))
results$Cp <- results$RSS / sigma2 - n + 2 * (results$p + 1)
results$AIC <- n * log(results$RSS / n) + 2 * (results$p + 1)
results$BIC <- n * log(results$RSS / n) +
  log(n) * (results$p + 1)
print(c(df_full = df_full, sigma2 = sigma2))
print(round(results[, -1], 4))

fit_gain <- n * log(rss["A"] / rss["B"])
cp_gain <- (rss["A"] - rss["B"]) / sigma2
print(c(fit_gain = fit_gain, cp_gain = cp_gain,
       AIC_penalty = 2, BIC_penalty = log(n), Cp_penalty = 2))
```

Question 5

- Step 2 is the first misuse because the final test set is used to compare models. The minimum test MSE tends to favor a model that performed unusually well by chance, making the reported error optimistic.
- Split the 370 training observations into five folds and calculate the average validation MSE for each predictor count, fitting on the other four folds each time. Select the smallest average, refit that model on all 370 training observations, and evaluate it once on the untouched final test set.
- The eleven test results can still be reported transparently as exploratory results, but their minimum is not an independent final assessment. A new independent test set or genuinely future observations would be needed for a fresh final evaluation.